

SmartFactory: AI-Powered Orchestration

Comprehensive Technical Reference & Viva Voce Examination Guide

Project Focus: Decentralized Multi-Attribute Auction Dispatch, Dynamic A* Collision-Free Pathfinding, and Real-Time Digital Twin Telemetry for Smart Factory Automated Guided Vehicles (AGVs/AMRs).

Architecture: Closed-Loop Autonomous Pipeline (Sense → Understand → Predict → Decide → Act → Update).

1. Technology Stack Breakdown

The application is built on an enterprise-grade full-stack architecture combining an asynchronous backend tick-engine with a reactive, hardware-accelerated HTML5 Canvas frontend.

Layer	Technology	Architectural Role & Description
Backend Framework	Python 3.10+, FastAPI	Asynchronous ASGI framework managing REST API endpoints, task lifecycles, and persistent client streaming.
Web Server	Uvicorn (ASGI)	High-performance lightning-fast server running the event loop and serving WebSocket client connections.
Concurrency	asyncio Background Tasks	Runs the continuous 150ms factory tick loop non-blockingly alongside concurrent HTTP requests.
Pathfinding Engine	A* (A-Star) Algorithm	Custom heuristic grid pathfinder calculating shortest paths with dynamic narrow corridor congestion penalties.
Dispatch Mechanism	Multi-Attribute Auction	Decentralized bidding evaluating travel distance, battery risk, corridor traffic, and future bottleneck impact.
Real-Time Stream	WebSockets (/ws/factory)	Full-duplex bidirectional stream broadcasting full state snapshots and discrete event stream at ~7-10 Hz.
Database	SQLite 3 (marketfloor.db)	Persistent storage for completed task records, auction logs, telemetry, and aggregated analytics.
Frontend Core	HTML5, Vanilla JavaScript (ES6+)	Zero-dependency reactive client managing multi-view tabs (Floor, Task Manager, Reports).
Visual Graphics	HTML5 Canvas API (2D Context)	Hardware-accelerated 60 FPS rendering of 40x30 factory grid, moving AGVs, stations, and heatmaps.
Styling System	Vanilla CSS3 (Design Tokens)	Custom CSS variables, responsive grid/flexbox, Warm Industrial palette, and print formatting (@media print).

2. Mathematical Formulation: Multi-Attribute Auction

MarketFloor Composite Bid Formula:

$$\text{Final Bid} = \text{TravelCost} + \text{BatteryRisk} + \text{CongestionPenalty} + \text{FutureImpact} - \text{Fairness}$$

- **Travel Cost (T):** Manhattan / A* distance: $|x_{src} - x_{agv}| + |y_{src} - y_{agv}| + \text{distance}(\text{src}, \text{dst})$
- **Battery Risk (B):** Exponential penalty when battery drops below 30%: $\max(0, 30 - \text{battery}) \times 1.5$
- **Congestion Penalty (C):** Sum of traffic congestion scores across intermediate cells: $\sum \text{congestion}(\text{cell})$
- **Future Impact (FI):** Heuristic predicting downstream bottleneck creation on critical machines/stations
- **Fairness Adjustment (F):** Priority discount granted to underutilized AGVs to balance wear and tear

3. Essential Viva Voce Questions & Model Answers

Categorized technical questions covering architecture, algorithms, networking, and safety.

Category 1: Project Overview & Core Motivation

Q1. What is the fundamental problem this project addresses?

Traditional AGV/AMR dispatching relies on centralized nearest-neighbor algorithms. While straightforward, nearest-neighbor creates major bottlenecks: it ignores downstream spatial congestion, overworks nearby vehicles causing uneven battery wear, and lacks resilience when vehicles fail or corridors jam. Our system implements a decentralized multi-attribute auction model that optimizes fleet-wide balance, bottleneck avoidance, and energy efficiency.

Q2. What is the 'Closed-Loop Autonomous Trace' and how does it function?

It represents the 6-stage Industry 4.0 autonomous feedback cycle implemented in our simulation: (1) **SENSE:** Collect IoT telemetry (machine vibration, optical bin sensors, battery levels). (2) **UNDERSTAND:** Assess operational risk (thermal anomalies, stock buffer shortages). (3) **PREDICT:** Forecast corridor traffic and buffer overflows before they materialize. (4) **DECIDE:** Execute multi-attribute auction to select the lowest-impact vehicle. (5) **ACT:** Generate collision-free A* routes and dispatch vehicles. (6) **UPDATE:** Synchronize the digital twin and commit audit records to SQLite.

Q3. How does MarketFloor differ from traditional AGV dispatching systems?

Traditional dispatch systems ask: 'Which AGV is physically closest right now?' MarketFloor asks: 'Which AGV can complete this transport mission with the minimum systemic disruption to future high-priority jobs?' MarketFloor may intentionally select an AGV slightly farther away if dispatching the closest vehicle would cause gridlock in a narrow corridor.

Category 2: Algorithms, Pathfinding & Math

Q4. Why did you choose A* Search over Dijkstra's Algorithm or BFS?

BFS is unweighted and incapable of evaluating dynamic congestion costs. Dijkstra's explores nodes equally in all directions, wasting computation on a 40×30 grid. **A* Search** utilizes the heuristic function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact accumulated path cost (including cell congestion) and $h(n)$ is the admissible **Manhattan distance**. This guarantees the optimal route while directing search towards the goal with minimal node expansions.

Q5. How does the system handle real-time corridor blockages and rerouting?

When a corridor blockage or disruption occurs (simulated via `sim.block_corridor()`), the traversal cost of those grid cells is elevated or set to non-walkable. The simulation checks all active AGVs whose planned route intersects the blocked cells and immediately re-runs A* pathfinding from their current position to the destination, seamlessly navigating around the bottleneck.

Q6. How is the multi-attribute auction decided among the AGVs?

When a task is created, a bidding request is broadcast. Each active, non-failed AGV evaluates the task and computes: $\text{Final Bid} = \text{TravelCost} + \text{BatteryRisk} + \text{CongestionPenalty} + \text{FutureImpact} - \text{Fairness}$. Bids are collected and sorted. The AGV with the **lowest composite bid** wins the mission. In case of a tie, battery level breaks the tie.

Category 3: System Architecture & Concurrency**Q7. Why did you choose FastAPI and Uvicorn over Flask or Django?**

FastAPI is built natively on ASGI (Asynchronous Server Gateway Interface) and Starlette. It supports asynchronous concurrency (async/await), high request throughput comparable to Node.js/Go, automatic JSON serialization via Pydantic, and native WebSocket support within the exact same process without requiring external workers or Redis.

Q8. How does the simulation tick loop run without blocking HTTP and WebSocket requests?

The tick loop is spawned as an asynchronous task (`asyncio.create_task(tick_loop())`) inside FastAPI's lifespan context. Because each tick execution ends with a non-blocking `await asyncio.sleep(interval)`, control is voluntarily yielded back to the Python event loop, allowing incoming HTTP and WebSocket messages to process concurrently without lag.

Q9. What database did you use and how is telemetry persisted?

We use **SQLite3** (`marketfloor.db`). It maintains auctions and `completed_tasks` tables. To prevent redundant disk writes during high-frequency ticks, the engine maintains in-memory sets (`saved_task_ids`, `saved_auction_ids`). Only newly completed missions trigger an INSERT OR REPLACE statement.

Category 4: Real-Time Networking & Frontend Engineering**Q10. Why use WebSockets instead of REST Polling or Server-Sent Events (SSE)?**

HTTP Polling introduces heavy HTTP header overhead and latency intervals. SSE is strictly unidirectional. **WebSockets** establish a single persistent, bidirectional TCP connection. The backend pushes full factory snapshots 7-10 times per second, and the frontend can transmit instantaneous commands (E-Stop, Task Assign, Disruptions) with sub-millisecond latency.

Q11. How does the canvas render 40×30 grid cells and moving vehicles smoothly at 60 FPS?

Instead of instantiating 1,200+ DOM nodes, we utilize the **HTML5 2D Canvas API**. Canvas operations utilize GPU acceleration. Upon receiving a state message, the canvas clears and redraws stations, corridors, route lines, and vehicle sprites in a single frame execution (~2ms).

Q12. What do the different visual AGV vehicle colors represent?

- **Blue (#0284c7)**: Default / Idle standby state.
- **Orange (#ea580c)**: Actively performing a transport mission (en route to pickup).
- **Grey (#6b7280)**: Carrying cargo and delivering to the target bay.
- **Green (#16a34a)**: Recharging docked at a charging pad.
- **Black (#1a1a1a)**: Assigned to a specialized Quality Check transport task.
- **Red (#dc2626)**: Mechanical fault / breakdown state.

Category 5: Fault Tolerance, Safety & Edge Cases

Q13. What happens if an AGV experiences a mechanical breakdown during a mission?

The vehicle's status changes to FAILED and its speed drops to 0. The assigned task status immediately transitions to RE_AUCTIONING. The engine automatically triggers a re-auction among surviving fleet members. A new AGV wins and is dynamically rerouted to recover the cargo and finish delivery.

Q14. How does Emergency Stop (E-Stop) differ from normal Pause/Stop?

Normal Stop pauses the animation clock and can be toggled resume at any time. **Emergency Stop** is an industrial safety interlock. It immediately zeros all vehicle velocities, halts all auction bidding, rejects new assignments, and flags an Attention alert across telemetry cards. Normal resume buttons are locked until the E-Stop is explicitly cleared.

Q15. How would you scale this system to 100+ AGVs and real factory hardware?

- (1) **Hardware Integration:** Connect FastAPI to physical vehicles using ROS 2 (Robot Operating System) or MQTT protocol.
- (2) **Reinforcement Learning:** Replace the heuristic future-impact scorer with a trained Deep Q-Network (DQN) trained on simulated traffic.
- (3) **Time-Series Database:** Migrate SQLite to PostgreSQL with TimescaleDB for high-throughput industrial telemetry logging.

Key Viva Presentation Strategy & Defense Highlights:

1. **Demonstrate the Paradigm Shift:** Emphasize that traditional dispatch is proximity-first (myopic), whereas MarketFloor is systemic-first (preventing future corridor deadlock before allocating vehicles).
2. **Highlight the Closed Loop:** Walk the examiner through Sense → Understand → Predict → Decide → Act → Update.
3. **Show Fault Tolerance Live:** Trigger 'AGV Failure' or 'Corridor Blockage' during your viva demo to show dynamic A* rerouting and automated task re-auctioning in action.
4. **Color Semantics:** Remember: Blue = Idle, Orange = Task Performing, Grey = Delivering, Green = Charging, Black = Quality Check, Red = Fault.